CSC 8840 - Homework 2: Sheep-Grass System

Student: Cherisha Killari Giribabu **Package:** Homework2026.PredatorPrey.CKillariGiribabu

Model design

The simulation uses a 30 x 30 wrapped cellular space. Every position is a DEVS atomic cell with one of three high-level conditions: empty, grass, or sheep. Cells communicate only through the eight neighbor input and output ports. The coupled model creates all 900 cells, their neighbor connections, and the display. The GlobalRef object holds shared cell conditions, cell references, a seeded random number generator, and four editable time parameters.

Empty: An empty cell passivates and has no internal event. On a grass message it becomes grass; on a sheep movement or birth message it becomes sheep. **Grass:** A grass cell schedules reproduction after grassReproduceT. At that event it selects at most one empty neighbor at random, sends a grass message, and schedules its next reproduction. If there are no empty neighbors, nothing grows. Grass is consumed as soon as a sheep arrives.

Sheep: A sheep has three clocks: toMoveTime, toDieTime, and toReproduceTime. Its next internal DEVS phase is chosen from the smallest remaining clock. When time passes, all three clocks decrease by that amount (sigma for an internal event and e for an external event). Moving takes sheepMoveT; the target is selected when the movement finishes. Grass neighbors have priority, then empty neighbors. A successful move sends a sheep message containing its remaining lifetime and reproduction time, resets the movement clock, and leaves the source cell empty. Entering grass resets the destination sheep lifetime. If no free target exists, the sheep stays and tries movement again later. Reproduction sends one birth message to a neighboring non-sheep cell, unless all are sheep. When the lifetime timer expires, the sheep dies.

Timing and simultaneous events

All three sheep timers run concurrently rather than sequentially. The event scheduler uses the earliest remaining time as sigma. Internal events subtract sigma from every timer; external events subtract e. Confluent messages are handled after the internal transition. A death wins any tie with movement or reproduction. A target cell is reserved when the outgoing event occurs, not at the start of a movement; this prevents two sheep from choosing the same destination at the same simulation instant. The reservation is released by the receiving cell.

Display and random choices

The simulation draws empty cells white, grass green, and sheep red using the provided newCellGridPlot display object. One global Random object with a fixed seed is used so identical parameters and scenarios can be compared. The wrapped grid makes each cell have eight neighbors, including corner and edge cells.

Scenarios

Scenario	Initial arrangement
1	One grass at center; no sheep
2	Four grass cells in separate locations
3	One sheep at center; no grass
4	Four sheep cells; no grass
5	Two neighboring grass cells plus one adjacent sheep
6	Randomly distributed grass and sheep populations

Assumptions and test observations

New sheep are born with full movement, lifetime, and reproduction clocks. Reproduction can place a new sheep in an adjacent grass cell, consuming that grass. An attempted sheep move that finds no available grass or empty neighbor does not reset the lifetime clock. Events occurring at exactly the same time are resolved with death priority in one cell and first-reservation priority across target cells.

A compilation check against the supplied DEVSJAVA source completed successfully. In sample runs of 350 simulator events, scenario 2 filled the grid with grass, while scenarios 4 and 6 ended with neither sheep nor grass. With 300 events, scenario 5 also ended with no organisms. These are sample observations, not proof of balanced oscillation. More parameter trials are needed; increasing sheepLifeT or changing grassReproduceT and sheepReproduceT may change survival and population cycles. The four parameters are deliberately left editable in GlobalRef.java so they can be tested independently.